Midas q2 Contracts Audit

Côme du Crest

2026-08-04

Table of contents

- Table of contents
- Midas q2 Contracts Audit
  - Scope
  - Context
  - Status
  - Legal Information And Disclaimer
- Issues
  - [Fixed][High] Zero `avgRate` ignored
  - [Fixed][High] Storage layout conflicts of redemption vaults
  - [Fixed][High] `mTokenPermissioned` does not check greenlist on burn
  - [Fixed][Med] Discrepancy in between specification and implementation
  - [Fixed][Low] MidasTimelockManager operation status can be hi-jacked
  - [Fixed][Low] `amountTokenWithoutFee` not truncated in deposit vault calculation
  - [Acknowledged][Info] Weak slippage protection on requests
  - [Acknowledged][Info] Users are not reimbursed on rejected requests
  - [Acknowledged][Info] DepositVaultWithUSTB crashes on failed transfer
  - [Info] DepositVaultWithUSTB only invests on instant transfers and not requests
  - [Acknowledged][Info] RedemptionVaultWithMToken assumptions
  - [Acknowledged][Info] mToken clawback could be used to bypass blacklist if execution flow is diverted
  - [Acknowledge][Info] `MidasTimelockManager` cannot schedule the same operation twice in a row
  - [Acknowledged][Info] Vault average rate bypasses variation tolerance
  - Optimisations and miscellaneous

Midas q2 Contracts Audit

This document presents the findings of a smart contract audit conducted by Côme du Crest.

For any inquiry contact me by email at come.ducrest@gmail.com or via telegram @ComeduCrest.

Scope

The scope include updates contracts of PR#226 of midas-apps/contracts. the last commit reviewed in this audit is 0x54f65e1.

The contracts have last been audited as of commit 0xff4d994 and the diff with this commit is the main focus of this review.

The analysed contracts are:

- DepositVault.sol: reworked DepositVault
- DepositVaultWithAave.sol: DepositVault that auto invests tokens into Aave
- DepositVaultWithMToken.sol: DepositVault that auto invests tokens into Midas
- DepositVaultWithMorpho.sol: DepositVault that auto invests tokens into Morpho
- DepositVaultWithUSTB.sol: DepositVault that auto invests tokens into USTB
- RedemptionVault.sol: reworked RedemptionVault
- RedemptionVaultWithAave.sol: RedemptionVault that pulls liquidity from Aave
- RedemptionVaultWithMToken.sol: RedemptionVault that pulls liquidity from Midas
- RedemptionVaultWithMorpho.sol: RedemptionVault that pulls liquidity from Morpho
- RedemptionVaultWithUSTB.sol: RedemptionVault that pulls liquidity from USTB
- abstract/ManageableVault.sol: reworked shared logic in between deposit and redemption vaults
- abstract/WithSanctionsList.sol: minimal changes
- abstract/MidasInitializable.sol: minimal changes
- mToken.sol: reworked mToken
- mTokenPermissioned.sol: minimal changes
- access/MidasAccessControl.sol: reworked access control

- access/MidasAccessControlTimelockController.sol: new contract inherits from OZ TimelockControllerUpgradeable
- access/MidasTimelockController.sol: new contract inherits from OZ TimelockController
- access/MidasTimelockManager.sol: new contract to implement timelock on authorized calls
- access/MidasPauseManager.sol: new contract to pause contracts or functions
- access/WithMidasAccessControl.sol: reworked with role checking modifiers
- access/Blacklistable.sol: minimal changes
- access/Greenlistable.sol: minimal changes
- libraries/MidasAuthLibrary.sol: new library
- libraries/PauseGuardsLibrary.sol: new library
- libraries/RateLimitLibrary.sol: new library
- libraries/RedemptionSwapperHelpersLibrary.sol: new library
- feeds/CompositeDataFeed.sol: minimal changes
- feeds/CompositeDataFeedMultiply.sol: minimal changes
- feeds/CustomAggregatorV3CompatibleFeed.sol: minimal changes
- feeds/CustomAggregatorV3CompatibleFeedAdjusted.sol: new minimal contract
- feeds/CustomAggregatorV3CompatibleFeedGrowth.sol: minimal changes
- feeds/DataFeed.sol: minimal changes
- misc/acre/AcreAdapter.sol: minimal changes (but require to check context)
- misc/adapters/BandStdChailinkAdapter.sol: minimal changes

Changes on Interfaces are expected to be insignificant and were not be directly audited. Other misc changes are simple solidity version upgrades and are not included in scope. Mocks and testers contracts are not in scope.

The code has been updated to fix reported issues. It has been reviewed as of commit `0x737a781` of `PR#249`.

Context

The most notable changes are the timelock controller that enforces a timelock on authorized calls through `MidasAccessControl` and the rewrite of vaults. A `Q2_2026_CONSOLIDATED_SPEC` markdown document has been provided and conformance with it has been analyzed.

Status

The report has been sent to the protocol team.

Only one [Info] issue is awaiting feedback, the rest has been fixed or acknowledged.

Legal Information And Disclaimer

1. This report is based solely on the information provided by Midas (the “Company”), with the assumption that the information provided is authentic, accurate, complete, and not misleading as of the date of this report. The auditor has not conducted any independent enquiries, investigations or due diligence in respect of the Company, its business or its operations.
2. Changes to the information contained in the documents, repositories and any other materials referenced in this report might affect or change the analysis and conclusions presented. The auditor is not responsible for monitoring, nor will we be aware of, any future additions, modifications, or deletions to the audited code. As such, the auditor does not assume any responsibility to update any information, content or data contained in this report following the date of its publication.
3. This report does not address, nor should it be interpreted as addressing, any regulatory, tax or legal matters, including but not limited to: tax treatment, tax consequences, levies, duties, data privacy, data protection laws, issues relating to the licensing of information technology, intellectual property, money laundering and countering the financing of terrorism, or any other legal restrictions or prohibitions. The auditor disclaims any liability for omissions or errors in the findings or conclusions presented in this report.
4. The views expressed in this report are solely my views regarding the specific issues discussed within this report. This report is not intended to be exhaustive, nor should it be construed as an assurance, guarantee or warranty that the code is free from bugs, vulnerabilities, defects or deficiencies. Different use cases may carry different risks, and integration with third-party applications may introduce additional risks.
5. This report is provided for informational purposes only and should not be used as the basis for making investment or financial decisions. This report does not constitute investment research and should not be viewed as an invitation, recommendation, solicitation or offer to subscribe for or purchase any securities, investments, products or services. The auditor is not a financial advisor, and this report does not constitute financial or investment advice.
6. The statements in this report should be considered as a whole, and no individual statement should be extracted or referenced independently.
7. To the fullest extent permitted by applicable laws, the auditor disclaims any and all other liability, whether in contract, tort, or otherwise, that may arise from this report or the use thereof.

Issues

[Fixed][High] Zero `avgRate` ignored

Summary

When deposit / redeem requests are approved using an average rate, if the calculated average rate is 0 then the rate is discarded and the admin-provided rate is used instead.

A calculated average rate of 0 should signify that too many tokens were instantly minted/redeemed by the user initially and none should be further minted/redeemed. Instead the user enjoys additional tokens for free.

Vulnerability Detail

The function `DepositVault._approveRequest()` computes the `avgRate` and only override the `newOutRate` if it is not 0:

```
    function _approveRequest(
        uint256 requestId,
        uint256 newOutRate,
        bool isSafe,
        bool isAvgRate
    )
        private
        returns (
            bool /* success */
        )
    {
        Request memory request = mintRequests[requestId];

        ...

        if (isAvgRate) {
            uint256 avgRate = _calculateHoldbackPartRateFromAvg(
                request,
                newOutRate
            );

            if (avgRate != 0) {
                newOutRate = avgRate;
            }
        }

        require(newOutRate > 0, InvalidNewMTokenRate());

        uint256 amountMToken = _quoteMTokenFromRequest(request, newOutRate);

        ...
```

```
32      }
```

The function `_calculateHoldbackPartRateFromAvg()` returns 0 if the amount of tokens minted in the instant part is greater than the target amount of tokens dictated by the admin input `avgMTokenRate`:

```
1       function _calculateHoldbackPartRateFromAvg(
2           Request memory request,
3           uint256 avgMTokenRate
4       ) internal pure returns (uint256) {
5           if (
6               avgMTokenRate == 0 ||
7               request.tokenOutRate == 0 ||
8               request.depositedInstantUsdAmount == 0
9           ) {
10              return 0;
11          }
12
13          uint256 targetTotalMTokenValue = ((request.depositedUsdAmount +
14              request.depositedInstantUsdAmount) * (10**18)) / avgMTokenRate;
15
16          uint256 instantPartMTokenValue = (request.depositedInstantUsdAmount *
17              (10**18)) / request.tokenOutRate;
18
19          if (targetTotalMTokenValue <= instantPartMTokenValue) {
20              return 0;
21          }
22
23          uint256 holdbackPartValue = targetTotalMTokenValue -
24              instantPartMTokenValue;
25
26          return (request.depositedUsdAmount * (10**18)) / holdbackPartValue;
27      }
```

If no token should be additionally minted to the user because they minted enough (or too many) in their instant deposit then the computed `avgRate = 0` will be ignored.

The issue is analogous for redeem requests in redemption vaults.

Impact

Under the specific condition where users minted/redeemed more tokens in the instant part of their requests than targeted by the average rate provided by the admin the user will receive tokens they should not receive.

Code Snippets

https://github.com/midas-apps/contracts/blob/54f65e1885319042ba170dc916b566b9ba7b181a/contracts/RedemptionVault.sol#L475

https://github.com/midas-apps/contracts/blob/54f65e1885319042ba170dc916b566b9ba7b181a/contracts/DepositVault.sol#L623

Recommendation

If average rate is 0 mark the request as processed and do not send any more tokens to the user.

Response

This has been fixed by requiring that average rate is not zero if the request has a non-zero `depositedInstantUsdAmount`. This means that instead of giving free tokens to the user a request with too many instantly minted tokens will revert when approved.

[Fixed][High] Storage layout conflicts of redemption vaults

Summary

I was informed that vaults would be redeployed and not upgraded. However, `RedemptionVaultWithUSTB` and `RedemptionVaultWithMToken` present signs via variables naming or documentation that they would be deployed and serve as an upgrade of existing vaults.

By comparing the storage layouts with the previous commit I have available (`0xff4d994`) I can see multiple storage layout conflicts in `RedemptionVault`.

Vulnerability Detail

`RedemptionVaultWithUSTB` has the following storage variable that seem to occupy storage slots to align the next ones for compatibility:

```
contract RedemptionVaultWithUSTB is RedemptionVault {
    ...
    uint256 private __deprecatedStorageSlot1;

    ...
    uint256 private __deprecatedStorageSlot2;
    ...
}
```

`RedemptionVaultWithMToken` has the following documentation:

> - @dev Storage layout is preserved for safe upgrades from RedemptionVaultWithSwapper

Impact

If vaults are upgraded from vaults deployed as of commit `0xff4d994` to the new `RedemptionVaultWithUSTB` or `RedemptionVaultWithMToken` storage layout conflicts will result in undefined behaviours.

Code Snippets

https://github.com/midas-apps/contracts/blob/54f65e1885319042ba170dc916b566b9ba7b181a/contracts/RedemptionVaultWithUSTB.sol#L31

https://github.com/midas-apps/contracts/blob/54f65e1885319042ba170dc916b566b9ba7b181a/contracts/RedemptionVaultWithMToken.sol#L19

Recommendation

Acknowledge the vaults will be redeployed and delete the confusing documentation / storage variables or analyse the storage layout of existing contracts to make sure they match the new contracts.

Response

Vaults will be redeployed and the storage variables and comment have been deleted.

[Fixed][High] `mTokenPermissioned` does not check greenlist on burn

Summary

In the `burn()` function of `mToken` there is a check to ensure `from` is not blacklisted. The inheriting `mTokenPermissioned` inherits that check but does not enforce that the `from` address be greenlisted and the `_beforeTokenTransfer()` does not enforce the greenlist if `to == address(0)` which is the case for a burn.

Vulnerability Detail

The `mToken.burn()` function enforces the blacklist:

```
    function burn(address from, uint256 amount)
        external
        onlyRoleNoTimelock(burnerRole(), false)
    {
        _onlyNotBlacklisted(from);
        _burn(from, amount);
    }
```

The `mTokenPermissioned._beforeTokenTransfer()` function ignores the greenlist in case of a burn:

```
    function _beforeTokenTransfer(
        address from,
        address to,
        uint256 amount
    ) internal virtual override(mToken) {
        if (to != address(0)) {
            if (!_inClawback && from != address(0)) {
                _onlyGreenlisted(from);
            }

            _onlyGreenlisted(to);
        }

        mToken._beforeTokenTransfer(from, to, amount);
    }
```

There is no overriding function in `mTokenPermissioned` to enforce the greenlist during a burn.

Impact

The burn function can be used to ignore the greenlist. This has been identified previously in the case of LZ or Axelar integrations to allow cross-chain transfers from an unauthorized address.

Code Snippets

https://github.com/midas-apps/contracts/blob/54f65e1885319042ba170dc916b566b9ba7b181a/contracts/mTokenPermissioned.sol#L61

Recommendation

Implement an override `burn()` function in `mTokenPermissioned` that enforces the greenlist.

Response

Fixed by implementing the recommendation.

[Fixed][Med] Discrepancy in between specification and implementation

[Acknowledged] Data feeds cannot be paused

The provided Q2 spec file states:

> Aggregators & DataFeeds should also integrate the pause manager (§3.3). Outstanding gap — see compliance report C1.

I am not sure what this "Outstanding gap" stands for but data feeds and aggregators are not pausable as there are no references to `requireNotPaused()` in any of them.

Response The team decided not to integrate the pause manager into the feeds/aggregators

[Fixed] `pausedAt` not stored in `MidasTimelockManager`

The spec states:

> On pause store: pausedAt, pausedBy, reasonCode (emitted), resolutionCouncilVersion = current-CouncilVersion

However the code never stores `pausedAt`:

```
    function pauseOperation(bytes32 operationId, uint8 pauseReasonCode)
        external
        onlyRoleNoTimelock(TIMELOCK_OPERATION_PAUSER_ROLE, false)
    {
        require(_isPendingOperation(operationId), OperationNotPending());

        (
            TimelockOperationStatus status,
            TimelockOperationDetails storage opDetails
        ) = _getOperationStatus(operationId);

        require(
            status == TimelockOperationStatus.Pending,
            UnexpectedOperationStatus(status)
        );

        uint256 councilVersion = securityCouncilVersion;
        opDetails.status = TimelockOperationStatus.Paused;
        opDetails.pauseReasonCode = pauseReasonCode;
        opDetails.councilVersion = councilVersion;
        opDetails.pauser = msg.sender;

        emit PauseTimelockOperation(
            msg.sender,
```

```
25              operationId,
26              pauseReasonCode,
27              councilVersion
28          );
29      }
```

[Acknowledged] No function to get `isExecutable(id)` in `MidasTimelockManager`

The spec states:

> getProposal(id) (impl getOperationDetails), getUnresolvedProposalIds() (impl getPendingOperations), isExecutable(id) (impl isFunctionReadyToExecute).

But there is no function `isExecutable()` that would return a boolean on wether the function operation is executable.

Response That could be retrieved from the timelock controller

[Fixed] `MidasTimelockManager.executeTimelockOperation()` does not follow CEI

The spec states:

> `execute()` must set state to Executed before the external call (CEI) and be nonReentrant

But `executeTimelockOperation()` does not follow CEI even though it is non re-entrant.

[Acknowledged] Delays live in `MidasAccessControl` and not `MidasTimelockManager`

The spec states:

> The existing central MidasAccessControl is extended (not replaced) to provide granular, function-level roles and per-role timelocks. Timelock delay values live in a separate MidasTimelockManager so they are configured independently of the role-granting logic (see §2).

However timelock delays are set and live in `MidasAccessControl` and not `MidasTimelockManager`.

Response Delays initially lived in the timelock manager, but as we added a functions that grants the role and sets the initial delay it needed to move to the ac, so specs were incorrect there

[Acknowledged] Average rate requests revert for instant requests

The spec states:

> passing isAvgRate for a non-holdback request reverts.

It also states:

> Edge case: when a request had instantPercentage == 0 (no instant first leg), the AvgRate functions (SafeBulkApproveAvgRate, …AtSavedRate, …AtCurrentRate) must remain functional (process as standard) — they must not revert. (See compliance report H1: currently reverts; needs fixing.)

Which seems to contradict itself.

Response "passing isAvgRate for a non-holdback request reverts" is a deprecated statement, we decided to allow approving regular requests trough the AvgRate fns as we likely grant our OPS team access only to those function, not to regular approve fns

[Acknowledged] FIFO request processing

The spec states:

> Example: next-to-process = 1; safebulk([1,2,3]) then safebulk([4,6,5]) → 1,2,3,4 process; 6 skipped (out of order); 5 skipped; ops re-initiates safebulk([5,6,…]).

However, with this exemple the implementation will process 1,2,3,4,5 and ignore 6.

Response It's incorrect in specs, it should just skip if value is out of order, but shouldn't invalidate all the values that go after it.

Impact

Some of these discrepancies may be legal requirements and need to be fixed. Others can be acknowledged and the spec can be updated.

Recommendation

Acknowledge the issues and update the spec or implement missing features.

[Fixed][Low] MidasTimelockManager operation status can be hi-jacked

Summary

The function `MidasTimelockManager.executeTimelockOperation()` does not respect the check-effect-interaction pattern and states:

> updating state after execution to be able to verify tx against current context, in case of reentrancy timelock.execute will revert

However, other functions that update the state can be entered such as `pauseOperation()`, `voteForVeto()`, `voteForExecution()`, and/or `abortOperation()` during an operation execution without causing reverts.

As a result, the read operation status during execution cannot be trusted or relied on.

Vulnerability Detail

The function `executeTimelockOperation()` first checks that the operation can be executed. Then it executes the operation and after that it updates the state for the operation:

```
    function executeTimelockOperation(address target, bytes calldata data)
        external
        nonReentrant
        onlyContractAdminNoTimelock(true)
    {
        ...

        require(
            status == TimelockOperationStatus.Pending ||
                status == TimelockOperationStatus.ReadyToExecute,
            UnexpectedOperationStatus(status)
        );

        require(
            _timelock.isOperationReady(operationId),
            TimelockOperationNotReady()
        );

        _timelock.execute(target, 0, data, bytes32(0), bytes32(dataHashIndex));

        _resetPendingSetCouncilOperation(opDetails);

        // updating state after execution to be able to verify tx against
            current context
        // in case of reentrancy timelock.execute will revert
        opDetails.status = TimelockOperationStatus.Executed;
        dataHashIndexes[dataHash] = dataHashIndex + 1;
        --proposerPendingOperationsCount[opDetails.operationProposer];
```

```
        require(_pendingOperations.remove(operationId), OperationNotPending());

        ...
    }
```

As an example `pauseOperation()` can be used at the beginning of the operation execution to transition the operation to the `TimelockOperationStatus.Paused` during operation execution:

```
    function pauseOperation(bytes32 operationId, uint8 pauseReasonCode)
        external
        onlyRoleNoTimelock(TIMELOCK_OPERATION_PAUSER_ROLE, false)
    {
        require(_isPendingOperation(operationId), OperationNotPending());

        (
            TimelockOperationStatus status,
            TimelockOperationDetails storage opDetails
        ) = _getOperationStatus(operationId);

        require(
            status == TimelockOperationStatus.Pending,
            UnexpectedOperationStatus(status)
        );

        uint256 councilVersion = securityCouncilVersion;
        opDetails.status = TimelockOperationStatus.Paused;
        opDetails.pauseReasonCode = pauseReasonCode;
        opDetails.councilVersion = councilVersion;
        opDetails.pauser = msg.sender;

        emit PauseTimelockOperation(
            msg.sender,
            operationId,
            pauseReasonCode,
            councilVersion
        );
    }
```

The code executed after `_timelock.execute(target, 0, data, bytes32(0), bytes32(dataHashIndex));` in `executeTimelockOperation()` will not revert and the operation will appear paused during part of its execution.

Impact

Operation status (among other state) cannot be relied on during operation execution. This is a read-only re-entrancy issue whose impact depend on state is being consumed by other contracts.

Code Snippets

https://github.com/midas-apps/contracts/blob/54f65e1885319042ba170dc916b566b9ba7b181a/contracts/access/MidasTimelockManager.sol#L271

Recommendation

Move the operation execution at the end of `executeTimelockOperation()`. Alternatively, if you need to rely on `opDetails.status` during the execution check that is still `TimelockOperationStatus.Pending` after `_timelock.execute()`.

Response

The issue has been fixed by respecting check-effect-interaction pattern. Note that `dataHashIndexes[dataHash] = dataHashIndex + 1;` is still after the operation execution but it has little impact.

[Fixed][Low] `amountTokenWithoutFee` not truncated in deposit vault calculation

Summary

The function `DepositVault._calcAndValidateDeposit()` uses user-controlled input `amountToken` to compute `result.amountTokenWithoutFee` and does not truncate the result to token decimals. As a result any transfer of tokens using this value may fail if the resulting amount is not round and the token has less than 18 decimals.

Vulnerability Detail

The function `DepositVault._calcAndValidateDeposit()` does not truncate `result.amountTokenWithoutFee`:

```
    function _calcAndValidateDeposit(
        address user,
        address tokenIn,
        uint256 amountToken,
        bool isInstant
    ) internal returns (CalcAndValidateDepositResult memory result) {
        ...

        (uint256 amountInUsd, uint256 tokenInUSDRate) = _convertTokenToUsd(
            tokenIn,
            amountToken
        );
        result.tokenAmountInUsd = amountInUsd;
        result.tokenInRate = tokenInUSDRate;
        address userCopy = user;

        _requireAndUpdateAllowance(tokenIn, amountToken);

        result.feeTokenAmount = _truncate(
            _getFeeAmount(_getFee(userCopy, tokenIn, isInstant), amountToken),
            result.tokenDecimals
        );

        result.amountTokenWithoutFee = amountToken - result.feeTokenAmount;  //
             @audit not truncated to token decimals if token decimals < 18

        ...
    }
```

This value is used in `_depositInstant()` to transfer tokens to the receiver:

```
    function _depositInstant(
        address tokenIn,
        uint256 amountToken,
        uint256 minReceiveAmount,
        address recipient
```

```
    ) internal virtual returns (CalcAndValidateDepositResult memory result) {
        address user = msg.sender;

        result = _calcAndValidateDeposit(user, tokenIn, amountToken, true);

        ...

        _instantTransferTokensToTokensReceiver(
            tokenIn,
            result.amountTokenWithoutFee + result.feeTokenAmount,
            result.tokenDecimals
        );

        ...
    }
```

Token transfers will fail if value is not truncated correctly:

```
    function _tokenTransferFromTo(
        address token,
        address from,
        address to,
        uint256 amount,
        uint256 tokenDecimals
    ) internal returns (uint256 transferAmount) {
        if (amount == 0) return 0;
        transferAmount = amount.convertFromBase18(tokenDecimals);
        uint256 truncatedAmount = transferAmount.convertToBase18(tokenDecimals)
            ;

        require(
            amount == truncatedAmount,
            InvalidRounding(amount, truncatedAmount)
        );

        ...
    }
```

Impact

Deposits with tokens that have less than 18 decimals may fail due to incorrect truncation.

Code Snippets

https://github.com/midas-apps/contracts/blob/54f65e1885319042ba170dc916b566b9ba7b181a/contracts/DepositVault.sol#L760

Recommendation

Truncate `result.amountTokenWithoutFee` to the token decimals.

Response

amountToken from input is expected to be already truncated, so now using it for transfer

[Acknowledged][Info] Weak slippage protection on requests

Summary

The slippage protection on deposit and redeem requests relies on variation tolerance that can be bypassed by admin under normal circumstances.

This may be intended behaviour.

Vulnerability Detail

During a deposit request, the user may specify a `minReceiveAmountInstantShare` for the minimum amount of mTokens to receive for the instant part. However, for the delayed part the slippage relies on storing `tokenOutRate` the rate used at the time of request and checking that the admin-provided rate at the time of approval is not too far from it:

```
    function _approveRequest(
        uint256 requestId,
        uint256 newOutRate,
        bool isSafe,
        bool isAvgRate
    )
        private
        returns (
            bool /* success */
        )
    {
        Request memory request = mintRequests[requestId];

        _validateRequest(requestId, request.recipient, request.status);
        _validateUserAccess(request.recipient, false);

        if (isSafe) {
            _requireVariationTolerance(request.tokenOutRate, newOutRate);
        }

        ...
    }
```

This variation tolerance check is performed only when `isSafe` is true and this value is controlled by the admin. Additionally, the value `variationTolerance` used to check the new rate is not "too far" from the instant rate can be updated by the admin at any point.

```
    function _requireVariationTolerance(uint256 prevPrice, uint256 newPrice)
        internal
        view
    {
        uint256 priceDif = newPrice >= prevPrice
            ? newPrice - prevPrice
```

```
            : prevPrice - newPrice;

        uint256 priceDifPercent = (priceDif * ONE_HUNDRED_PERCENT) / prevPrice;

        require(
            priceDifPercent <= variationTolerance,
            PriceVariationExceeded(priceDifPercent, variationTolerance)
        );
    }
```

The issue is analogues for RedemptionVault's redeem requests.

Impact

Users are weakly protected for slippage in between their request and its approval.

Code Snippets

https://github.com/midas-apps/contracts/blob/54f65e1885319042ba170dc916b566b9ba7b181a/contracts/DepositVault.sol#L614

https://github.com/midas-apps/contracts/blob/54f65e1885319042ba170dc916b566b9ba7b181a/contracts/RedemptionVault.sol#L466

Recommendation

Acknowledge that this is intended behaviour or add an additional parameter to requests for minimum of expected total token.

Response

Intended by design

[Acknowledged][Info] Users are not reimbursed on rejected requests

Summary

The deposits and redemption vaults do not reimburse tokens to the request recipient when a request is rejected.

Vulnerability Detail

The function `DepositVault.rejectRequest()` validates the request and mark it is cancelled without reimbursing the request initiator / recipient. There is no function that allow a request recipient to withdraw tokens from a cancelled request.

```
    function rejectRequest(uint256 requestId) external onlyContractAdmin {
        Request memory request = mintRequests[requestId];

        _validateRequest(requestId, request.recipient, request.status);
        _validateAndUpdateNextRequestIdToProcess(requestId, true);

        mintRequests[requestId].status = RequestStatus.Canceled;

        upcomingSupply -= _quoteMTokenFromRequest(
            request,
            request.tokenOutRate
        );

        emit RejectRequest(requestId);
    }
```

The situation is the same for `RedemptionVault`.

Impact

There may be an out of contract mechanism to reimburse users in case of cancelled request and this may be intended behaviour.

Code Snippets

https://github.com/midas-apps/contracts/blob/54f65e1885319042ba170dc916b566b9ba7b181a/contracts/DepositVault.sol#L270

https://github.com/midas-apps/contracts/blob/54f65e1885319042ba170dc916b566b9ba7b181a/contracts/RedemptionVault.sol#L267

Recommendation

Acknowledge the issue or implement a function to allow cancelled requests to be withdrawn by their recipient / original initiator.

Response

It’s intended design, same as in currently live contracts.

[Acknowledged][Info] DepositVaultWithUSTB crashes on failed transfer

Summary

The function `DepositVaultWithUSTB._instantTransferTokensToTokensReceiver()` documents that:

> in case of USTB deposits are disabled or invest token is not supported by USTB, it will act as the original transfer …

However, in case USTB cannot be invested the function may revert, blocking the transfer and not falling back to the default behaviour.

Vulnerability Detail

The function `_instantTransferTokensToTokensReceiver()` has multiple `require()` that may cause a revert if USTB deposits cannot be performed (e.g. token not supported by USTB or `subscribe` call fails):

```
    /**
     * @dev overrides original transfer to tokens receiver function
     * in case of USTB deposits are disabled or invest token is not supported
     * by USTB, it will act as the original transfer
     * otherwise it will take payment tokens from user, invest them into USTB
     * and will transfer USTB to tokens receiver
     *
     * @param tokenIn token address
     * @param amountToken amount of tokens to transfer in base18
     * @param tokensDecimals decimals of tokens
     */
    function _instantTransferTokensToTokensReceiver(
        address tokenIn,
        uint256 amountToken,
        uint256 tokensDecimals
    ) internal virtual override {
        if (!ustbDepositsEnabled) {
            return
                super._instantTransferTokensToTokensReceiver(
                    tokenIn,
                    amountToken,
                    tokensDecimals
                );
        }

        ISuperstateToken.StablecoinConfig memory config = ISuperstateToken(ustb
            )
            .supportedStablecoins(tokenIn);

        require(
```

```
            config.sweepDestination != address(0),
            UnsupportedUSTBToken(tokenIn)
        );

        require(config.fee == 0, USTBFeeNotZero(config.fee));

        address ustbToken = ustb;
        uint256 transferredAmount = _tokenTransferFromUser(
            tokenIn,
            address(this),
            amountToken,
            tokensDecimals
        );

        IERC20(tokenIn).safeIncreaseAllowance(ustbToken, transferredAmount);
        ISuperstateToken(ustbToken).subscribe(
            tokensReceiver,
            transferredAmount,
            tokenIn
        );
    }
```

Impact

Documented function behaviour seem incorrect even though the likelihood of the function reverting may be tiny if USTB is properly configured.

Code Snippets

https://github.com/midas-apps/contracts/blob/54f65e1885319042ba170dc916b566b9ba7b181a/contracts/DepositVaultWithUSTB.sol#L126

Recommendation

Acknowledge that this is intended behaviour and updated documentation or adapt the function so that it does not revert and fallback to the default behaviour instead.

Response

It’s intended design, same as in currently live contracts.

[Info] DepositVaultWithUSTB only invests on instant transfers and not requests

Summary

Contrary to similar vaults (e.g. `DepositVaultWithAave`), the `DepositVaultWithUSTB` only invests received tokens from instant deposits and not deposits requests.

Vulnerability Detail

The contract `DepositVaultWithUSTB` implements `_instantTransferTokensToTokensReceiver()` that is only called by the inherited `DepositVault` during instant deposits. It does not implement `_requestTransferTokensToTokensReceiver()`

```
    function _instantTransferTokensToTokensReceiver(
        address tokenIn,
        uint256 amountToken,
        uint256 tokensDecimals
    ) internal virtual override ...

    function _depositInstant(
        address tokenIn,
        uint256 amountToken,
        uint256 minReceiveAmount,
        address recipient
    ) internal virtual returns (CalcAndValidateDepositResult memory result) {
        ...

        _instantTransferTokensToTokensReceiver(
            tokenIn,
            result.amountTokenWithoutFee + result.feeTokenAmount,
            result.tokenDecimals
        );

        ...
    }
```

Impact

Tokens are only invested during instant deposits and not after requests approvals. This may be intended behaviour.

Code Snippets

https://github.com/midas-apps/contracts/blob/54f65e1885319042ba170dc916b566b9ba7b181a/contracts/DepositVaultWithUSTB.sol#L126

Recommendation

Acknowledge the issue or adapt the `DepositVaultWithUSTB` to also invest tokens on request approval.

[Acknowledged][Info] RedemptionVaultWithMToken assumptions

Summary

`RedemptionVaultWithMToken` assumes that underlying redemption vault mToken is not stable, and that the vault uses the same data feed for tokenOut rate.

Vulnerability Detail

The `RedemptionVaultWithMToken` uses `getSwapperDetails()` to get the mTokenARate which uses `vault.mTokenDataFeed().getDataInBase18()`:

```
    function getSwapperDetails(
        IRedemptionVault _redemptionVault,
        address _getBalanceOf
    )
        internal
        view
        returns (
            uint256 mTokenARate,
            IERC20 mTokenA,
            uint256 mTokenABalance
        )
    {
        mTokenARate = _redemptionVault.mTokenDataFeed().getDataInBase18();
        mTokenA = IERC20(address(_redemptionVault.mToken()));
        mTokenABalance = mTokenA.balanceOf(_getBalanceOf);
    }
```

However, if the mToken is stable then its rate on the underlying vault is overridden to 1 and the data feed is ignored:

```
    function _getMTokenRate() internal view returns (uint256 mTokenRate) {
        mTokenRate = _getTokenRate(address(mTokenDataFeed), false);
        _validateTokenRate(mTokenRate);
    }

    function _getTokenRate(address dataFeed, bool stable)
        internal
        view
        virtual
        returns (uint256)
    {
        // @dev if dataFeed returns rate, all peg checks passed
        uint256 rate = IDataFeed(dataFeed).getDataInBase18();

        if (stable) return STABLECOIN_RATE;

        return rate;
    }
```

Additionally, the token out rate used for computing the amount of mTokenA to redeem is the one provided as input which is the one of the current vault (`address(this)`) and not of the underlying vault.

```
    function _obtainVaultLiquidity(
        address tokenOut,
        uint256 missingAmountBase18,
        uint256 tokenOutRate,
        uint256, /* currentTokenOutBalanceBase18 */
        uint256 tokenOutDecimals
    )
        internal
        virtual
        override
        returns (
            uint256 /* obtainedLiquidityBase18 */
        )
    {
        IRedemptionVault _redemptionVault = redemptionVault;

        (
            uint256 mTokenARate,
            IERC20 mTokenA,
            uint256 mTokenABalance
        ) = RedemptionSwapperHelpersLibrary.getSwapperDetails(
                _redemptionVault,
                address(this)
            );

        // Ceil so the inner vault's floored output is still >=
            missingAmountBase18.
        uint256 mTokenAAmount = Math.mulDiv(
            missingAmountBase18,
            tokenOutRate,
            mTokenARate,
            Math.Rounding.Up
        );

        ...
    }
```

Impact

If mToken of the underlying vault is stable or it uses a different rate for tokeOut then the calculation will be incorrect and an incorrect amount of tokenOut will be redeemed.

I assume this is unlikely to happen and has a low impact.

Code Snippets

https://github.com/midas-apps/contracts/blob/54f65e1885319042ba170dc916b566b9ba7b181a/contracts/RedemptionVaultWithMToken.sol#L122

Recommendation

Acknowledge that this situation cannot occur or verify that it cannot occur in `_obtainVaultLiquidity()`

Response

mToken rate is never stable and payment tokens are usually constant across all the vaults

[Acknowledged][Info] mToken clawback could be used to bypass blacklist if execution flow is diverted

Summary

The clawback mechanism allows mToken admins to perform transfers ignoring the blacklist for the `from` address. This is done by setting an `_inClawback` flag before the transfer and unsetting it after. If any hook that are executed during the transfer call an external code, the mToken transfer function could be re-entered to profit from this `_inClawback` state.

Vulnerability Detail

The function `clawback()` sets the `_inClawback` flag before calling `_transfer()`:

```
    function clawback(uint256 amount, address from) external onlyContractAdmin
        {
        _inClawback = true;
        _transfer(from, clawbackReceiver, amount);
        _inClawback = false;
    }
```

This flag is checked in `_beforeTokenTransfer()` to bypass the blacklist on `from`:

```
    function _beforeTokenTransfer(
        address from,
        address to,
        uint256 amount
    ) internal virtual override(ERC20PausableUpgradeable) {
        PauseGuardsLibrary.requireNotPaused(accessControl, msg.sig);

        if (to != address(0)) {
            if (!_inClawback && from != address(0)) {
                _onlyNotBlacklisted(from);
            }
            _onlyNotBlacklisted(to);
        }

        ...
    }
```

If any hook of mToken transfers (in `_beforeTokenTransfer()` or `_afterTokenTransfer()` or in the called access control library) yield execution to an external contract, that external contract will be able to bypass the blacklist during a clawback.

Impact

It is not currently exploitable as no external contracts are called during authorization checks in `_beforeTokenTransfer()` or `_afterTokenTransfer()` but is something to keep in mind when developing new contracts inheriting from mToken.

Code Snippets

https://github.com/midas-apps/contracts/blob/54f65e1885319042ba170dc916b566b9ba7b181a/contracts/mToken.sol#L227

Recommendation

Acknowledge the issue.

Reponse

Acknowledge the issue as it is not exploitable right now.

[Acknowledge][Info] `MidasTimelockManager` cannot schedule the same operation twice in a row

Summary

The function `MidasTimelockManager._scheduleTimelockOperation()` cannot be used twice with the same target and data before the first operation has been aborted or executed.

Vulnerability Detail

`_scheduleTimelockOperation()` gets the operationId and later requires that is has not already been added to the pending operation list:

```
    function _scheduleTimelockOperation(address target, bytes calldata data)
        private
    {
        ...

        (
            bytes32 operationId,
            bytes32 dataHash,
            uint256 dataHashIndex
        ) = _getOperationId(_timelock, target, data);

        ...

        require(_pendingOperations.add(operationId), OperationAlreadyPending())
            ;

        ...
    }
```

The `operationId` is computed as a hash of the target, data, and an incremental counter `dataHashIndexes[dataHash]`:

```
    function _getOperationId(
        TimelockController _timelock,
        address target,
        bytes calldata data
    )
        private
        view
        returns (
            bytes32 operationId,
            bytes32 dataHash,
            uint256 dataHashIndex
        )
    {
        dataHash = _getDataHash(target, data);
        dataHashIndex = dataHashIndexes[dataHash];
```

```
16
17          operationId = _timelock.hashOperation(
18              target,
19              0,
20              data,
21              bytes32(0),
22              bytes32(dataHashIndex)
23          );
24      }
```

However, that counter is only increased on operation execution / abortion and not when scheduled.

Impact

The same operation cannot be schedule twice without the first operation being handled. This prevents scenario like recurrent operations that should be scheduled every day with a delay longer than a day.

Code Snippets

https://github.com/midas-apps/contracts/blob/54f65e1885319042ba170dc916b566b9ba7b181a/contracts/access/MidasTimelockManager.sol#L664

https://github.com/midas-apps/contracts/blob/54f65e1885319042ba170dc916b566b9ba7b181a/contracts/access/MidasTimelockManager.sol#L278

https://github.com/midas-apps/contracts/blob/54f65e1885319042ba170dc916b566b9ba7b181a/contracts/access/MidasTimelockManager.sol#L403

Recommendation

Acknowledge the issue if this is intended behaviour. Otherwise, increment `dataHashIndexes[dataHash]` after it is consumed by `_scheduleTimelockOperation()`.

Response

For day-to-day operations we likely won't have a timelock or it will be set to a smaller value. In case if we will need to have recurent operations with timelock - this check could be bypassed by just appending random data to the end of the calldata as it would be ignored anyway. The onchain dataHash/dataHashIndex are mostly needed to simplify the interface and make scheduling more determenistic and less depended on the frontend

[Acknowledged][Info] Vault average rate bypasses variation tolerance

Summary

Deposit and redemption vaults use check that the admin input rate is not outside variation tolerance when used with `isSafe`. However, if average rate is used the applied rate may be overriden by a rate outside of the variation tolerance. This may leave the request recipient to receive less tokens than anticipated.

Vulnerability Detail

`RedemptionVault._approveRequest()` checks the variation tolerance on the new rate before overriding it when `isAvgRate` is true:

```
    function _approveRequest(
        uint256 requestId,
        uint256 newMTokenRate,
        bool isSafe,
        bool isAvgRate
    )
        private
        returns (
            bool /* success */
        )
    {
        Request memory request = redeemRequests[requestId];

        _validateRequest(requestId, request.recipient, request.status);

        _validateUserAccess(request.recipient, false);

        if (isSafe) {
            _requireVariationTolerance(request.mTokenRate, newMTokenRate);
        }

        if (isAvgRate) {
            uint256 avgRate = _calculateHoldbackPartRateFromAvg(
                request,
                newMTokenRate
            );

            if (avgRate != 0) {
                newMTokenRate = avgRate;
            }
        }
        ...
    }
```

The issue is analoguous in `DepositVault`

Impact

A request that is partly instant may have sent tokens to `recipientInstant` as provided in `redeemRequest()` with a high rate and sent tokens to `request.recipient` later at a rate that is below the variation tolerance.

If `recipientInstant` is different from `request.recipient` this can be seen as value theft from `request.recipient` to `recipientInstant`.

This is probably intended behaviour.

Code Snippets

https://github.com/midas-apps/contracts/blob/54f65e1885319042ba170dc916b566b9ba7b181a/contracts/DepositVault.sol#L614

https://github.com/midas-apps/contracts/blob/54f65e1885319042ba170dc916b566b9ba7b181a/contracts/RedemptionVault.sol#L466

Recommendation

Acknowledge and document the issue.

Response

It's an intended behavior as calculated avgRate could be much lower than the provided one so we cant compare it with the current price

Optimisations and miscellaneous

This part lists minor gas/code optimizations that shouldn't make the code less readable or improve overall readability. It also lists questions about unclear code segments.

[Incorrect] Approving a request does not check if contract is paused

The functions `_approveRequest()` of deposit and redemption vaults do not check wether the contract is paused when validating the user access. This may or may not be intended behaviour.

```
    function _approveRequest(
        uint256 requestId,
        uint256 newOutRate,
        bool isSafe,
        bool isAvgRate
    )
        private
        returns (
            bool /* success */
        )
    {
        Request memory request = mintRequests[requestId];

        _validateRequest(requestId, request.recipient, request.status);
        _validateUserAccess(request.recipient, false);
        ...
    }

    function _validateUserAccess(address user, bool validatePaused) ...
```

https://github.com/midas-apps/contracts/blob/54f65e1885319042ba170dc916b566b9ba7b181a/contracts/DepositVault.sol#L611

https://github.com/midas-apps/contracts/blob/54f65e1885319042ba170dc916b566b9ba7b181a/contracts/RedemptionVault.sol#L463

Response This is incorrect as `ManageableVault` overrides `_validateFunctionAccessWithTimelock()` to check for paused function.

[Acknowledged] `RedemptionVaultWithAave._obtainVaultLiquidity()` does not check contract balance

The documentation of `RedemptionVaultWithAave._obtainVaultLiquidity()` states:

> Check if contract has enough tokenOut balance for redeem; if not, withdraw the missing amount from the Aave V3 Pool

However the function does not check any balance and relies on caller to perform that check. The check is done in inherited `RedemptionVault._obtainLiquidity()`

https://github.com/midas-apps/contracts/blob/54f65e1885319042ba170dc916b566b9ba7b181a/contracts/RedemptionVaultWithAave.sol#L108

[Fixed] Unnecessary `onlyContractAdmin` modifier

The function `RedemptionVault.safeBulkApproveRequestAtSavedRate()` uses the `onlyContractAdmin` but it immediately calls the private `_safeBulkApproveRequest()` which also uses the `onlyContractAdmin` modifier meaning the whole authorization chain is checked twice which wastes gas.

This is also true for `DepositVault.safeBulkApproveRequestAtSavedRate()`.

https://github.com/midas-apps/contracts/blob/54f65e1885319042ba170dc916b566b9ba7b181a/contracts/RedemptionVault.sol#L212

https://github.com/midas-apps/contracts/blob/54f65e1885319042ba170dc916b566b9ba7b181a/contracts/DepositVault.sol#L213

[Fixed] Noop in `_safeBulkApproveRequest()`

The function `_safeBulkApproveRequest()` has a loop with a conditional `continue` at the end of the loop which is not doing anything (the loop continues anyway independently of the condition).

This is true for both `RedemptionVault` and `DepositVault`.

https://github.com/midas-apps/contracts/blob/54f65e1885319042ba170dc916b566b9ba7b181a/contracts/DepositVault.sol#L352

https://github.com/midas-apps/contracts/blob/54f65e1885319042ba170dc916b566b9ba7b181a/contracts/RedemptionVault.sol#L426

[Fixed] `__ManageableVault_init` could use true in `_validateAddress()`

The function `__ManageableVault_init` validates input addresses but does not check if addresses are self for `mToken` and `mTokenDataFeed`. I do not see a reason for this:

```
    function __ManageableVault_init(
        CommonVaultInitParams calldata _commonVaultInitParams
    ) internal onlyInitializing {
        __WithMidasAccessControl_init(_commonVaultInitParams.ac);
        __WithSanctionsList_init_unchained(
            _commonVaultInitParams.sanctionsList
        );

        _validateAddress(_commonVaultInitParams.mToken, false);
        _validateAddress(_commonVaultInitParams.mTokenDataFeed, false);
        ...
    }
```

https://github.com/midas-apps/contracts/blob/54f65e1885319042ba170dc916b566b9ba7b181a/contracts/abstract/ManageableVault.sol#L201

[Acknowledged] `mToken._inClawback` could use transient storage

The storage `_inClawback` is always set and unset within the same transaction and could use transient storage for readability and gas usage:

```
    function clawback(uint256 amount, address from) external onlyContractAdmin
        {
        _inClawback = true;
        _transfer(from, clawbackReceiver, amount);
        _inClawback = false;
    }
```

Response It would require changing the compilation target which i dont really want to do as we support a lot on “non-mainstream” chains that still may not support cancun yet.

[Acknowledged] Storage layout of mToken and feeds

The `mToken` contract adds storage variables and adjust the size of the `__gap` variable accordingly in comparison with previous storage layout as of commit `0xff4d994`. However, it adds a new `___gap` “to match with the gap of previous implementations” which I cannot explain.

The contracts `CustomAggregatorV3CompatibleFeedGrowth` and `DataFeed` also add such a gap.

https://github.com/midas-apps/contracts/blob/54f65e1885319042ba170dc916b566b9ba7b181a/contracts/mToken.sol#L84

https://github.com/midas-apps/contracts/blob/54f65e1885319042ba170dc916b566b9ba7b181a/contracts/feeds/DataFeed.sol#L54

https://github.com/midas-apps/contracts/blob/54f65e1885319042ba170dc916b566b9ba7b181a/contracts/feeds/CustomAggregatorV3CompatibleFeedGrowth.sol#L101

Response In q2 we removed the whole “products” folder that contained actuall contracts that we were deploying for a specific product, and each of those products contained a __gap on top of the gap in mtoken/feed/aggregator/etc. If we wont add those additional __gaps - upgrade from harhdat-upgrade fails as it flags that we removed a storage var, and to fix that we would need to disable storage checks which is not secure

[Fixed] `mToken.intializeV2()` initializes to version 3

The `initializeV2()` function of `mToken` uses `reinitializer(3)` instead of `reinitializer(2)` which initializes the contract to version 3. This can bring confusion for future upgrades.

```
    function initializeV2(address _clawbackReceiver)
        public
        virtual
        reinitializer(3)
        onlyProxyAdmin
    {
        ...
    }
```

[Fixed] `RateLimitLibrary._availableCapacity()` with 0 window

`RateLimitLibrary._availableCapacity()` considers a case where window is 0 but it cannot be 0 in any way.

```
    uint256 private constant _MIN_WINDOW = 1 minutes;

    function setWindowLimit(
        WindowRateLimits storage limits,
        uint256 window,
        uint256 limit
    ) internal returns (uint256 previousLimit) {
        if (limits.windows.add(window)) {
            require(window >= _MIN_WINDOW, WindowTooShort(window));
        }
        ...
    }

    function _availableCapacity(
        uint256 amountInFlight,
        uint256 lastUpdated,
        uint256 limit,
        uint256 window
```

```
    ) private view returns (uint256 inFlight, uint256 remaining) {
        uint256 elapsed = block.timestamp - lastUpdated;

        uint256 decay = Math.mulDiv(
            limit,
            elapsed,
            window > 0 ? window : 1,
            Math.Rounding.Down
        );

        ...
    }
```

https://github.com/midas-apps/contracts/blob/54f65e1885319042ba170dc916b566b9ba7b181a/contracts/libraries/RateLimitLibrary.sol#L212

[Fixed] `MAX_PENDING_OPERATIONS_PER_PROPOSER` is actually max of max

The variable `MidasTimelockManager.MAX_PENDING_OPERATIONS_PER_PROPOSER` is actually the limit of what the `maxPendingOperationsPerProposer` can be set to. So it is the `MAX_OF_MAX_PENDING_OPERATIONS_PER_PROPOSER`.

https://github.com/midas-apps/contracts/blob/54f65e1885319042ba170dc916b566b9ba7b181a/contracts/access/MidasTimelockManager.sol#L79

[Acknowledged] `MidasTimelockManager.voteForVeto()` can be called after `voteForExecution()`

The function `MidasTimelockManager.voteForExecution()` checks that the caller did not already vote for veto, likely to prevent double voting. However, the function `voteForVeto()` does not check whether caller already voted for execution and allows double voting. This may be intended behaviour.

https://github.com/midas-apps/contracts/blob/54f65e1885319042ba170dc916b566b9ba7b181a/contracts/access/MidasTimelockManager.sol#L321

Response It's a design choice, so when you voted for the execution you can still veto it if you change your mind, but not vice verca